

# SPI\_Slave Verification and RTL2GDS Flow

Koen Van Caekenberghe, ChipDesign B.V.

June 2026

## Contents

|          |                                   |          |
|----------|-----------------------------------|----------|
| <b>1</b> | <b>SPI Slave Description</b>      | <b>2</b> |
| 1.1      | Overview . . . . .                | 2        |
| 1.2      | SPI Protocol . . . . .            | 2        |
| 1.3      | Register Map . . . . .            | 2        |
| 1.4      | Port Interface . . . . .          | 2        |
| 1.5      | Internal State Machine . . . . .  | 3        |
| <b>2</b> | <b>RTL2GDS Flow</b>               | <b>3</b> |
| 2.1      | Key Flow Files . . . . .          | 3        |
| 2.2      | Running the Flow . . . . .        | 3        |
| 2.3      | Flow Execution Results . . . . .  | 3        |
| <b>3</b> | <b>Back-to-Back Verification</b>  | <b>4</b> |
| 3.1      | Methodology . . . . .             | 4        |
| 3.2      | Simulation Commands . . . . .     | 4        |
| 3.3      | Results . . . . .                 | 4        |
| 3.4      | RTL Simulation Waveform . . . . . | 5        |
| 3.5      | B2B Overlay Waveform . . . . .    | 5        |
| <b>4</b> | <b>Conclusions</b>                | <b>6</b> |

# 1 SPI Slave Description

## 1.1 Overview

The spi\_slave is a small, synchronous SPI slave IP block designed for use in the IHP SG13G2 process. It exposes eight 8-bit configuration and status registers to an SPI master and provides a secondary parallel read port for on-chip register readback independent of the SPI bus.

## 1.2 SPI Protocol

The core operates in **SPI Mode 0** (CPOL = 0, CPHA = 0): the master drives data on the falling edge of SCK and the slave samples on the rising edge.

Every transaction consists of two bytes:

1. **Command byte** — the MSB (bit 7) carries the R/W flag: 1 = Read, 0 = Write; bits [6:0] carry the 7-bit register address.
2. **Data byte** — for a write this is the value to store; for a read the slave shifts out the register value on MISO while MOSI is ignored.

## 1.3 Register Map

Eight 8-bit registers are addressable at addresses 0x00–0x07. A read at any address outside this range returns 0xFF. Reset (power-on) values are listed below.

| Address | Register  | Reset value | Address | Register  | Reset value |
|---------|-----------|-------------|---------|-----------|-------------|
| 0x00    | Register0 | 0x0A        | 0x04    | Register4 | 0x0E        |
| 0x01    | Register1 | 0x0B        | 0x05    | Register5 | 0x10        |
| 0x02    | Register2 | 0x0C        | 0x06    | Register6 | 0x20        |
| 0x03    | Register3 | 0x0D        | 0x07    | Register7 | 0x30        |

## 1.4 Port Interface

| Port      | Direction | Description                                                    |
|-----------|-----------|----------------------------------------------------------------|
| Clk       | in        | System clock; all internal logic is synchronous to this domain |
| iRST_N    | in        | Active-low asynchronous reset                                  |
| SCK       | in        | SPI clock from the master                                      |
| MOSI      | in        | Master-out slave-in data                                       |
| SSEL      | in        | SPI slave select, active low                                   |
| MISO      | out       | Master-in slave-out data                                       |
| Add2_in   | in        | Parallel read port: 8-bit register address                     |
| Data2_out | out       | Parallel read port: register data (updated every Clk cycle)    |
| debug     | out       | Sticky OR of state-machine states visited since last reset     |

## 1.5 Internal State Machine

The SPI interface logic is implemented as a Moore state machine with five states clocked by Clk. Asynchronous SPI inputs (SCK, MOSI, SSEL) are first captured through a two-flip-flop metastability synchroniser before entering the state machine.

| State   | Description                                              | debug bit |
|---------|----------------------------------------------------------|-----------|
| IDLE    | Waits for SSEL to go low                                 | 0         |
| COMMAND | Shifts in 8-bit command/address byte on SCK rising edges | 1         |
| WRITE   | Shifts in 8-bit data byte; writes register on completion | 2         |
| READ    | Shifts out register value on MISO on SCK falling edges   | 3         |
| END     | Waits for SSEL to return high, then returns to IDLE      | 4         |

## 2 RTL2GDS Flow

The repository includes a **LibreLane**-based RTL2GDS flow configured for the IHP SG13G2 PDK.

### 2.1 Key Flow Files

- flow/Makefile – entry point for LibreLane and OpenROAD commands.
- flow/config.yaml – LibreLane configuration file.
- flow/run\_flow.sh – environment setup and flow wrapper.
- flow/ihp\_pdk.env.example – example PDK environment template.
- flow/constraint.sdc – timing constraint for the spi\_slave clock.
- flow/place\_route.tcl – fallback OpenROAD placement/routing script.
- flow/spi\_slave\_synth.v – Yosys-generated structural Verilog netlist (used in B2B verification).

### 2.2 Running the Flow

```
cd spi_slave_github_repo/flow
cp ihp_pdk.env.example ihp_pdk.env
# Edit ihp_pdk.env to point to the local IHP PDK installation
./run_flow.sh
```

### 2.3 Flow Execution Results

The RTL2GDS flow was executed from flow/ and produced a completed LibreLane/OpenROAD run under runs/RUN\_2026-06-07\_13-47-22. Key outputs include:

- runs/RUN\_2026-06-07\_13-47-22/final/gds/spi\_slave.gds (803 KiB, includes seal ring)
- runs/RUN\_2026-06-07\_13-47-22/final/def/spi\_slave.def (624 KiB)

The flow ran all 69 steps and completed with warnings only (no errors). A patch was applied to the installed librelane seal-ring script to fix a KLayout API incompatibility with the PDK's Python-wrapper PCells: layout.create\_cell was replaced by add\_pcell\_variant and the die dimensions were corrected from database units (nm) to the  $\mu\text{m}$  values expected by the seal-ring PCell.

## 3 Back-to-Back Verification

### 3.1 Methodology

Back-to-back (B2B) verification confirms functional equivalence between the RTL source (`spi_slave.v`) and the Yosys-synthesised netlist (`spi_slave_synth.v`, in `flow/`). Both receive identical stimulus and their outputs are compared.

Three SPI transactions are exercised to confirm both read and write behaviour:

1. **Tx1 READ** Register 1 (address byte 0x81: R/W = 1, addr = 0x01). Expected on MISO: 0x0B (reset value). Verifies the read path.
2. **Tx2 WRITE** 0xFF to Register 3 (address byte 0x03: R/W = 0, addr = 0x03).
3. **Tx3 READ** Register 3 (address byte 0x83: R/W = 1, addr = 0x03). Expected on MISO: 0xFF. Confirms that Tx2 was committed to the register file.

Each testbench is self-checking: captured MISO bytes and the parallel-port output are compared against expected values with `$display` pass/fail reporting. The testbenches are compiled twice (RTL and synthesised netlist) using a `ifndef SYNTH` guard.

### 3.2 Simulation Commands

```
# RTL simulation (tb_spi_slave.v — self-checking, 3 transactions)
iverilog -g2012 -o tb_rtl.out tb_spi_slave.v
vvp tb_rtl.out          # PASS printed to stdout; produces tb_spi_slave.vcd

# B2B comparison — RTL
iverilog -g2012 -o tb_cmp_rtl.out tb_spi_slave_compare.v
vvp tb_cmp_rtl.out      # produces tb_spi_slave_compare.vcd

# B2B comparison — synthesised netlist (-DSYNTH skips 'include')
iverilog -g2012 -DSYNTH -o tb_cmp_syn.out \
    flow/spi_slave_synth.v tb_spi_slave_compare.v
vvp tb_cmp_syn.out      # produces tb_spi_slave_synth.vcd
```

### 3.3 Results

All nine observed signal values at the end of simulation ( $t = 1100\text{ ns}$ ) were identical between RTL and synthesised netlist:

| Signal                | RTL  | Synth | Status |
|-----------------------|------|-------|--------|
| <code>rst_n</code>    | 1    | 1     | MATCH  |
| <code>ssel</code>     | 1    | 1     | MATCH  |
| <code>sck</code>      | 0    | 0     | MATCH  |
| <code>mosi</code>     | 1    | 1     | MATCH  |
| <code>miso</code>     | 1    | 1     | MATCH  |
| <code>data</code>     | 0xFF | 0xFF  | MATCH  |
| <code>debug</code>    | 0x1F | 0x1F  | MATCH  |
| <code>miso_rx1</code> | 0x0B | 0x0B  | MATCH  |
| <code>miso_rx3</code> | 0xFF | 0xFF  | MATCH  |

miso\_rx1 = 0x0B confirms the Tx1 READ returned Register 1’s reset value. miso\_rx3 = 0xFF is the definitive write confirmation: Tx3 READ of Register 3 via SPI returned exactly the value written by Tx2. data = 0xFF shows the same value on the parallel read port (Add2\_in = 7 after the write). debug = 0x1F (bits 0–4 all set) confirms all five FSM states (IDLE, COMMAND, WRITE, READ, END) were visited.

### 3.4 RTL Simulation Waveform

Figure 1 shows the key SPI signals, parallel data output, and debug register over the full 1200 ns simulation. Three transactions are visible: Tx1 READ (green), Tx2 WRITE (orange), and Tx3 READ confirming the write (blue). data transitions from 0x0D to 0xFF at the end of Tx2 and holds that value through Tx3.

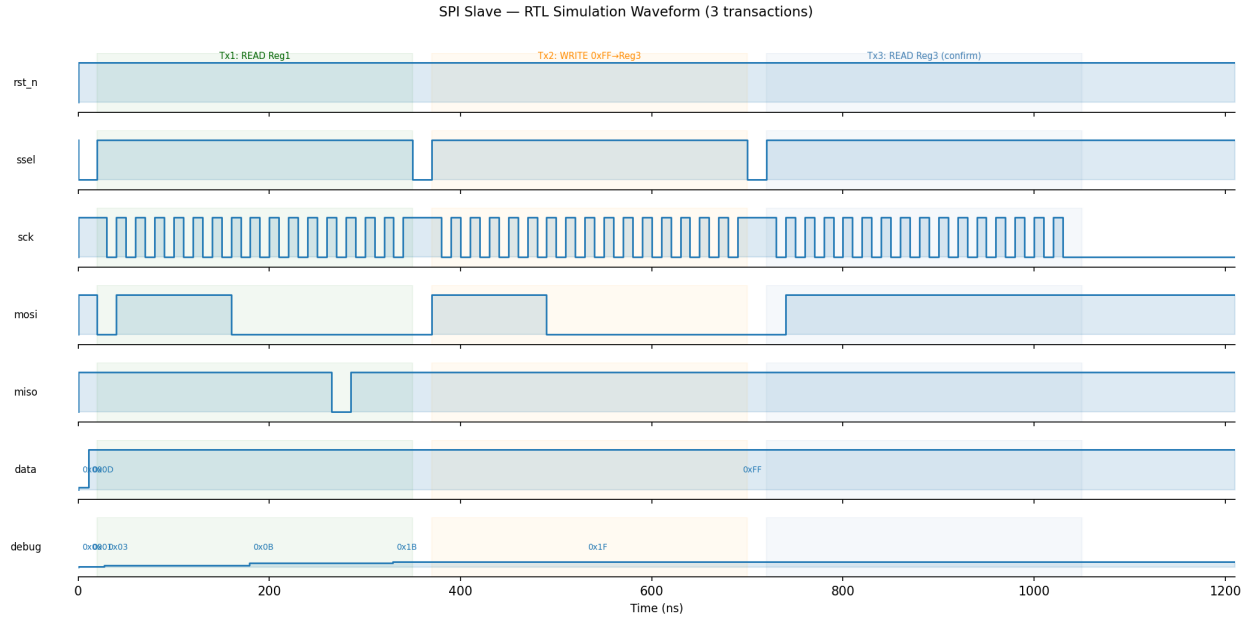


Figure 1: RTL simulation: key signals over the full testbench run.

### 3.5 B2B Overlay Waveform

Figure 2 overlays the RTL (blue, solid) and synthesised-netlist (red, dashed) waveforms for the output and captured-result signals. miso\_rx1 and miso\_rx3 are the bytes assembled by the testbench from the MISO stream. The traces are indistinguishable, confirming functional equivalence including the write-confirmation read-back.

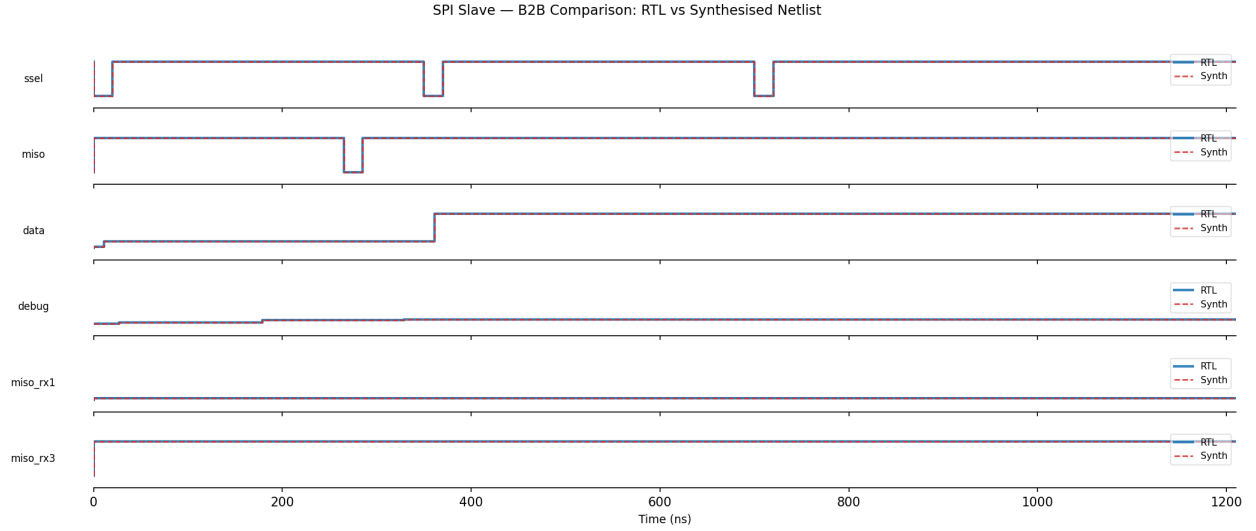


Figure 2: B2B overlay: RTL (blue solid) vs. synthesised netlist (red dashed). Traces overlap completely.

## 4 Conclusions

The spi\_slave design has been verified at two levels:

- **Functional correctness:** A register read (Tx1), a register write (Tx2), and a read-back of the written register (Tx3) were all exercised. The SPI read-back `miso_rx3 = 0xFF` directly confirms that Tx2 was committed to the register file.
- **Synthesis equivalence:** All observable output signals are bit-for-bit identical between the RTL model and the Yosys-generated structural netlist, confirming that synthesis introduced no functional changes.

The LibreLane/OpenROAD RTL2GDS flow has been executed successfully, yielding a final GDS for the IHP SG13G2 target process.